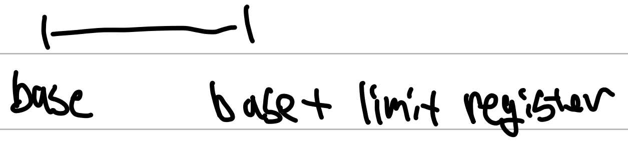
CH08 메모리 관리

- access protection


base
base+ limit register

- address binding 시점

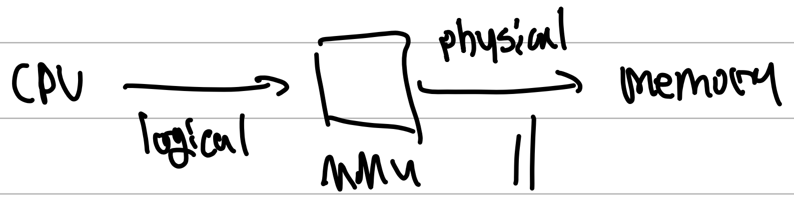
(컴파일) logical = physical
로드
실행 ≠

- logical / physical address

CPU에 의해 생성 (virtual)
메모리 상의 주소

- MMU 메모리 관리 유닛

실행시간에 논리주소를 물리 주소로 바인딩


CPU
logical
MMU
physical
memory
||

(logical + reallocating register 값)

〃 base register 값

- Dynamic linking   Static : linking 할때 다 합침

  실행시간 때까지 링킹연기

  stub 으로 루틴 주소의 명령 실행

- Swapping — page 단위   프로세스 단위

$$\text{memory} \overset{\text{out}}{\underset{\text{in}}{\rightleftarrows}} \text{backing store (disk)}$$

(roll-in 우선순위
roll-out 중)

보통 비활성화. (임계값 이상 도달시에만)
threshold

스위치 시간   100mb 프로세스, 50mb/초 전송률

→ swap out 시간은 2초 (2000 ms)

따라서 총 스와핑에 4초

(request_memory()
release_memory())

- contiguous allocation

  base ~ limit 범위체크, + reallocation register = physical address

  0   n

- multi-partition allocation

  사용가능한 곳 (hole)

어디에 넣을까? | first fit
best fit
worst fit

빈공간 합치기: compaction

- fragmentation
  - external 남는곳 있지만 연속적이지 않음 (hole 인데 사용 X)
  - internal 할당된 메모리이지만 안씀

first fit 사용시 N개 할당, 0.5 N개 fragment로 손실
1/3 사용불가 = 50% rule

- segmentation

프로그램은 segment의 집합 | 프로시저, 함수, 메소드, 객체
지역 전역 변수, 스택 ...

<segment-number, offset> → base 주소 + offset
↳ limit 보다 작아야함.

segment table base, limit 사용. 세그먼트의 개수 (segment table length register)
↑
segment table base register.

- paging

fixed-size (physical : frame
logical 에서 page)

page table

$$\underbrace{\overset{(p)}{\text{page number}}\ \vert\ \overset{(d)}{\text{page offset}}}_{m}$$

page number: $m-n$ | page offset: n

page size 2^n
주소 크기 2^m

메모리 2번 접근
↓ 해결
TLB (translation look-aside buffer) or associative memory
↳ 각 엔트리에 ASID 있는 경우 있음

wire down 으로 TLB에 페이지 고정 가능

- effective access time (그냥 넘어감)

lookup 시간 ε
Hit ratio α

$$EAT = \underbrace{(1+\varepsilon)\alpha}_{\text{메모리 access}} + \underbrace{(2+\varepsilon)(1-\alpha)}_{2\times\text{메모리 액세스}} = \underline{2+\varepsilon-\alpha}$$

2X 메모리 access 시간 - hit rate X 메모리 엑세스 시간

hit X 메모리 1번 + (1-hit) X 메모리 2번